

CS4227_CS5722 - Software Architecture

Designing the architecture and Evaluating a Component-Based Research Management System

Semester 2 AY 24/25

Assignment 3

Group members:

Shane Barden - 24033944

Ivor D Souza - 24177431

Manjeshwar Aniruddh Mallya – 24124133

David Parreño Barbuzano - 24196223

Rohan Sikder - 24165816

Date: 31/03/2025

Table of Contents

1. Summary
2. Architecture Evaluation
    a. Team Introduction
    b. Architecture Styles Overview
    c. Evaluation Approach
3. Scenario-Based Evaluation
    a. Evaluation Method
    b. Quality Attributes Assessment
    c. Tools & Implementation
4. Quality Attributes Assessed
5. Conclusion
6. Bibliography

Summary

Research Management Systems (RMS) are software platforms whose main objective is to provide various services for the management of research projects, data, resources, funding, and collaboration across academic, scientific, and industrial research environments. This document evaluates the architecture employed in the design and implementation of five RMS prototypes. The following sections provide a

detailed analysis of the architectural styles used, the evaluation approach, the assessed quality attributes, and the tools that support the evaluation process. Finally, the document concludes with a summary of the key insights obtained from the assessment process.

Architecture Evaluation

Team Introduction

The RMS prototypes were designed and implemented by five software engineers. Each of these prototypes provides users with different features that will be included in the final product. The decision of the architectures to use was conducted through a detailed analysis of the advantages and disadvantages of each architecture for each prototype. Specifically, the architectures that were considered are as follows: Event-driven Architecture, Client-server approach with MVC, Microservices architecture, Hexagonal Architecture, and Client-Server Layered architecture.

Architecture Styles Overview

First, the Event-Driven Architecture (https://github.com/rohansikder/RMS) was designed and developed by Rohan Sikder. This system is based on asynchronous communication between components, where events trigger specific actions within the system. The evaluation process would be conducted by David Parreño Barbuzano.

Second, the Client-server Architecture with MVC (https://github.com/losedavidpb/rms-citeProgress) was designed and developed by David Parreño Barbuzano. This architecture follows a two-tier model that divides the system into two primary components: the client and the server. It also incorporates the MVC design pattern, which divides the application into three distinct layers: the Model, the View, and the Controller. Rohan Sikder would be responsible for the assessment of David's architecture.

Third, the Microservices Architecture (https://github.com/ivor11/RMS) by Ivor D Souza, is designed to break down the RMS into a collection of loosely coupled, independently deployable services, each of them responsible for a specific business functionality. The evaluation process would be conducted by Manjeshwar Aniruddh Mallya.

Fourth, the Hexagonal Architecture (https://github.com/mallyaaniruddh/rmsZ/), designed and developed by Manjeshwar Aniruddh Mallya, uses a system where the core logic is isolated from external services

through well-defined interfaces, allowing easy integration with different external components. This architecture will be evaluated by Shane Barden.

Finally, the Client-Server Layered Architecture (https://github.com/Shaneb7102/ReactComponentLogin) was designed and developed by Shane Barden. This is a Client-Server architecture that organizes the system into distinct layers, where each layer has specific responsibilities. The evaluation will be conducted by Ivor D Souza.

Evaluation Approach

During the project assessment stage, the team utilized Architecture Tradeoff Analysis Method (ATAM) to assess architectural designs for various styles. ATAM is an organized and extensively applied method to assess software architectures based on key quality indicators including performance, security, modifiability, usability and accessibility. It permitted the team to systematically evaluate trade-offs, determine risks and assess the effect of different architectural decisions on system quality.

All team members met to go over their architectural designs at a group meeting. Many designs have been analyzed, and after very careful comparison one architecture was discovered to have clear benefits - particularly in terms of scalability and modifiability. Its attributes made it the optimal option for long-term system objectives and flexibility.

Using ATAM, the group managed to know more about the behavior of competing quality attributes in various architectures and how trade-offs impact the system robustness. The assessment discovered the advantages and drawbacks of each approach and provided the team with a deeper understanding into architectural concepts and better position itself for making sound design choices.

3. Scenario-Based Evaluation

3.1. Evaluation Method

For this evaluation, we apply the Architecture Trade-Off Analysis Method (ATAM) to assess five different architectural styles implemented for research management systems. As established by (Kazman, 2000),

ATAM is particularly appropriate for this comparison as it:

1. Systematically evaluates architectural decisions against quality attribute requirements
2. Identifies trade-offs between quality attributes

3. Discovers risks and sensitivity points in the architectures
4. Provides a structured approach to comparing multiple architectural alternatives

Our ATAM implementation follows these steps (Clements, 2002):

1. Present business drivers and quality attributes
2. Present architectural approaches
3. Generate and prioritize scenarios
4. Analyze architectural approaches using scenarios
5. Identify sensitivity points and trade-offs
6. Present results

Business Drivers

From analyzing the five architectural implementations, we identified these keys business drivers:

- Research Management: Ability to create, track, and manage research projects and proposals
- Collaboration: Support for user interactions, discussions, and sharing of research materials
- Integration: Connection with external research services (Google Scholar, Requirement Bazaar, Crossref)
- Authentication: Secure user authentication and role-based access control
- Document Management: Version control and management of research documents

Scenario Generation and Selection

We consolidated scenarios from each team member's evaluation to create a comprehensive scenario catalog. These scenarios represent critical use cases and quality attribute concerns:

From Shane's Hexagonal Architecture Evaluation:

S1: User Authentication Flow

- **Trigger:** A user attempts to log in to the RMS
- **Pre-conditions:** User has valid credentials in the system
- **Post-conditions:** User receives a JWT token stored in local storage
- **Expected Output:** User gains access to appropriate features based on their role
- **Quality Attributes:** Security, Usability

S2: Research Project Submission

- **Trigger:** A researcher submits a new research project
- **Pre-conditions:** User is authenticated with researcher privileges
- **Post-conditions:** Project is stored in the database and pending admin approval
- **Expected Output:** Confirmation message and project appears in researcher's dashboard
- **Quality Attributes:** Functionality, Reliability

S3: External API Data Integration

- **Trigger:** System fetches metadata for a research paper
- **Pre-conditions:** Valid research paper with identifiable metadata
- **Post-conditions:** External data enriches the research project information
- **Expected Output:** Combined internal and external data presented to the user
- **Quality Attributes:** Integration, Data Quality

S4: System Evolution and Adaptation

- **Trigger:** New external API needs to be integrated
- **Pre-conditions:** Existing system architecture with defined ports
- **Post-conditions:** New adapter implemented without changing core business logic
- **Expected Output:** System functionality extended without disrupting existing features
- **Quality Attributes:** Adaptability, Maintainability

From Rohan's MVC Evaluation:

S5: Proposal Submission

- **Trigger:** Researcher fills out and submits a form
- **Pre-conditions:** User is logged in
- **Post-conditions:** Proposal stored in memory, pending admin review
- **Expected Output:** Proposal appears in pending proposals list
- **Quality Attributes:** Usability, Maintainability

S6: Feedback Workflow

- **Trigger:** Admin approves or rejects a proposal
- **Pre-conditions:** Valid proposal ID selected
- **Post-conditions:** Proposal either converted to research or removed

- **Expected Output:** Research dashboard updates accordingly
- **Quality Attributes:** Security, Performance

S7: Filtering and Sorting

- **Trigger:** User applies a tag/date search filter
- **Pre-conditions:** Data loaded in frontend
- **Post-conditions:** Filtered list of results appears
- **Expected Output:** Instant update to UI with filtered proposals
- **Quality Attributes:** Usability, Performance

S8: Token Expiry or Invalid Login

- **Trigger:** Token is invalid/expired, or login fails
- **Pre-conditions:** User submits login request
- **Post-conditions:** Session rejected; user redirected
- **Expected Output:** Appropriate error message shown
- **Quality Attributes:** Security, Maintainability

From David's Event-Driven Evaluation:

S9: High Traffic Event Handling

- **Trigger:** Multiple users interact with the system simultaneously
- **Pre-conditions:** System is running with many users logged in
- **Post-conditions:** Notifications are delivered to users
- **Expected Output:** The system handles a high volume of events with minimal delay
- **Quality Attributes:** Performance, Scalability

S10: Rendering with Large Data Sets

- **Trigger:** A user posts a reply to a thread with many nested replies
- **Pre-conditions:** Many threads and replies already present
- **Post-conditions:** The UI updates to show the new reply
- **Expected Output:** UI remains responsive even with large data sets
- **Quality Attributes:** Usability, Performance, Scalability

S11: Extending Notification Types

- **Trigger:** A developer wants to add a new notification type
- **Pre-conditions:** Developer has access to the codebase
- **Post-conditions:** System handles the new notification type
- **Expected Output:** New notification type works without breaking existing functionality
- **Quality Attributes:** Maintainability

S12: Error Handling Challenges

- **Trigger:** An error occurs while sending a notification
- **Pre-conditions:** Notification system is active
- **Post-conditions:** System manages the error appropriately
- **Expected Output:** Error is logged, and user receives feedback
- **Quality Attributes:** Maintainability, Usability

From Aniruddh's Microservices Evaluation:

S13: Research Paper Search via External API

- **Trigger:** User inputs a search query
- **Pre-conditions:** Valid query, internet connectivity
- **Post-conditions:** Crossref API returns structured metadata
- **Expected Output:** Results displayed as cards containing metadata
- **Quality Attributes:** Integration, Usability

S14: Document Version Creation and Preview

- **Trigger:** Document edited and saved with metadata
- **Pre-conditions:** Valid input and metadata provided
- **Post-conditions:** Version stored in-memory
- **Expected Output:** Updated preview with author and timestamp
- **Quality Attributes:** Functionality, Modularity

S15: Frontend Service Modularity Simulation

- **Trigger:** User navigates between Document Control and Search
- **Pre-conditions:** Isolated rendering components
- **Post-conditions:** Smooth independent component rendering
- **Expected Output:** No interference between modules

- **Quality Attributes:** Modularity, Maintainability

S16: API Load and Rate Limiting

- **Trigger:** High volume of external API requests
- **Pre-conditions:** Peak usage scenario
- **Post-conditions:** Graceful handling of rate-limited API responses
- **Expected Output:** Informative feedback without UI failures
- **Quality Attributes:** Resilience, Performance, User Experience

From Ivor's Layered Architecture Evaluation:

S17: Concurrent User Authentication

- **Trigger:** 150 users attempt to log in simultaneously
- **Pre-conditions:** System under normal operation
- **Post-conditions:** Users receive authentication responses
- **Expected Output:** Response time remains under acceptable threshold
- **Quality Attributes:** Performance, Scalability

S18: Brute Force Attack Prevention

- **Trigger:** Attacker attempts multiple password guesses
- **Pre-conditions:** Valid username with repeated login attempts
- **Post-conditions:** System restricts further attempts
- **Expected Output:** Account lockout after threshold reached
- **Quality Attributes:** Security

S19: Adding Two-Factor Authentication

- **Trigger:** New requirement for enhanced security
- **Pre-conditions:** Existing authentication system
- **Post-conditions:** Two-factor authentication implemented
- **Expected Output:** Minimal code changes required
- **Quality Attributes:** Maintainability, Security

S20: New User Registration

- **Trigger:** New user with no prior experience attempts to register

- **Pre-conditions:** User accessing the registration page
- **Post-conditions:** User account created
- **Expected Output:** Clear, intuitive process with helpful error messages
- **Quality Attributes:** Usability

Scenario Analysis Process

For each selected scenario, we:

- Defined the scenario in detail with its stimulus, environment, and expected response
- Evaluated how each architecture would handle the scenario
- Rated each architecture's performance (Low, Medium, High)
- Assessed the risk level for each architecture
- Provided evidence from the documentation to support our assessment

3.2. Quality Attributes Assessment

This section presents the quality attributes assessment for each architecture designed. Essentially, the performance, security, maintainability, and usability of each prototype will be evaluated. Empirical evidence obtained using certain tools will be also included.

Event-driven

Performance

Concerning performance, the prototype demonstrates excellent loading speed, responsiveness, and stability. As shown in the Figure, it achieves a 97% score, indicating high optimization with efficient resource use and minimal delays. However, further improvements are possible. Based on the results obtained with Lighthouse, reducing unused JavaScript, eliminating render-blocking resources, and removing unnecessary CSS rules will enhance even more the performance of the system.

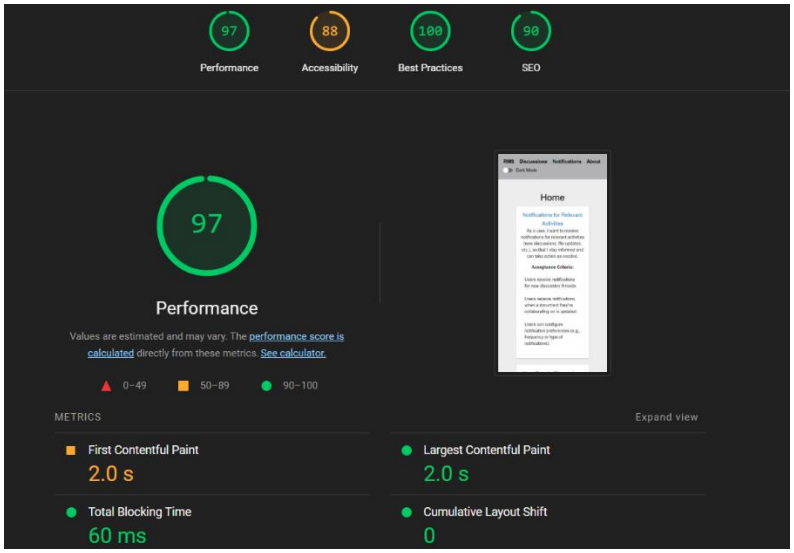


Figure 1 Performance of the Event-Driven Architecture

Security

As illustrated in the following figure, the Event-Driven prototype does not exhibit any security vulnerabilities, indicating that the system is secure, reliable, and adheres to best practices for web security. However, this may be due to the absence of a backend, meaning that critical processes such as authentication, data storage, and server-side processing have not been implemented. Additionally, a security hotspot detected by SonarQube should be considered, as it recommends verifying whether the use of *Math.random()* is safe for generating the ID of a new notification.



Figure 2 Security of the Event-Driven Architecture

Maintainability

Regarding the maintainability, the system is maintainable which means that the code is well-structured, readable, and easy to modify or extend. However, unlike the previous quality attributes, this prototype presents certain issues that should be fixed in future iterations. As illustrated in the following Figures, the system has 28 issues, 16 found in SonarQube and 12 detected with ESLint. Specifically, these issues are primarily related to code readability, structure, and best practices. The main concerns are:

1. **Variable Declarations & Loops:**
    - Missing let, const, or var keyword in variable declaration.
    - Expected a ‘for-of’ loop instead of a ‘for’ loop with this simple iteration.
    - Unexpected ‘any’ data type, specify another one.
2. **Component & Prop Management (React Best Practices):**
    - Props should be marked as read-only for immutability.
    - Components should be moved outside parent components and use props for data passing.
    - Avoid using array index as keys in React lists.
3. **Operator & Expression Optimization:**
    - Prefer ‘??’ instead of ‘||’ for safer handling of null or undefined values.
    - Use optional chaining for cleaner and more readable code.
4. **Function & Code Structure:**
    - Excessive function nesting (more than 4 levels deep).
5. **Deprecated Elements:**
    - Usage of deprecated elements (e.g., paragraph).

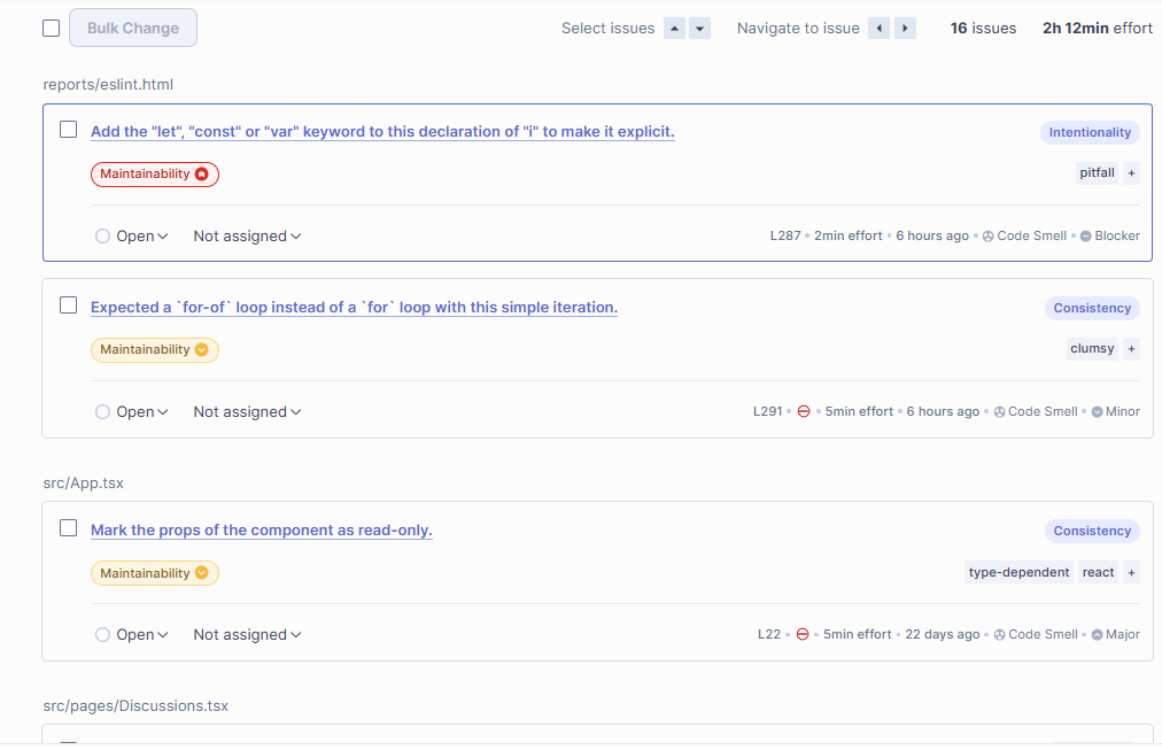


Figure 3 Maintainability of the Event-Driven Architecture

ESLint Report

12 problems (12 errors, 0 warnings) - Generated on Mar 29, 2025

[+] src\App.tsx 0 problems			
[+] src\main.tsx 0 problems			
[+] src\pages\About.tsx 0 problems			
[+] src\pages\Discussions.tsx 0 problems			
[+] src\pages\Home.tsx 0 problems			
[-] src\pages\NotificationSystem.tsx 4 problems (4 errors, 0 warnings)			
8:11	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
19:42	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
27:38	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
35:46	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
[-] src\pages\Notifications.tsx 5 problems (5 errors, 0 warnings)			
25:9	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
159:11	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
172:36	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
173:34	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
174:42	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
[-] src\utils\eventBus.ts 3 problems (3 errors, 0 warnings)			
6:16	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
8:14	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
10:22	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
[+] src\vite-env.d.ts 0 problems			

Figure 4 ESLint Report for the Event-Driven Architecture

Usability

After using the official website of this prototype, it can be said that it is highly usable, as it presents an intuitive and easy-to-navigate user interface. The well-structured layout, clear navigation paths, and user-friendly design contribute to a seamless experience. However, its accessibility could be improved, as highlighted in the following Figure. Enhancements such as ensuring that lists contain only <li> elements, and heading elements are in a sequentially descending order might further optimize the accessibility.

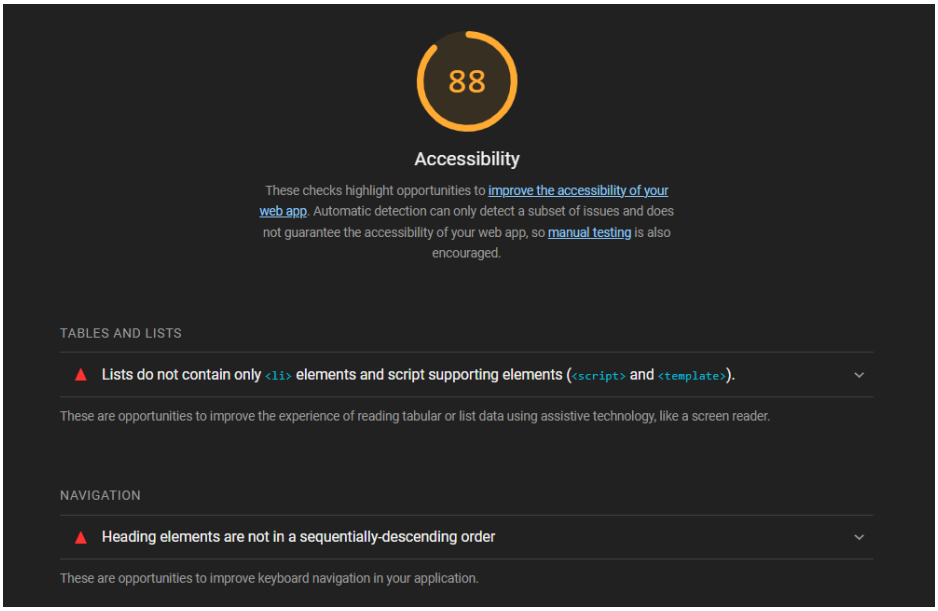


Figure 5 Accessibility of the Event-Driven Architecture

Client-Server + MVC

Performance

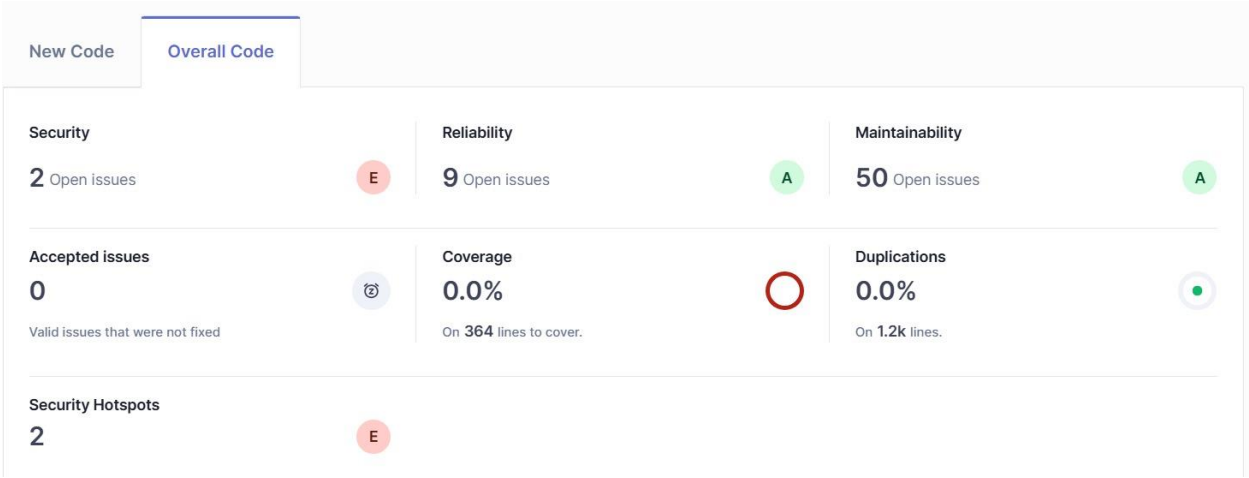
The MVC prototype demonstrates solid performance with smooth loading and stable interactions. While there is room for optimization, ESLint detected only one minor issue—an unexpected any type used in Filter.tsx. This suggests overall code efficiency and minimal performance bottlenecks. Optimizations like refining JavaScript usage and improving loop structures can boost responsiveness even further.

ESLint Report

1 problem (1 error, 0 warnings) - Generated on Mar 31, 2025

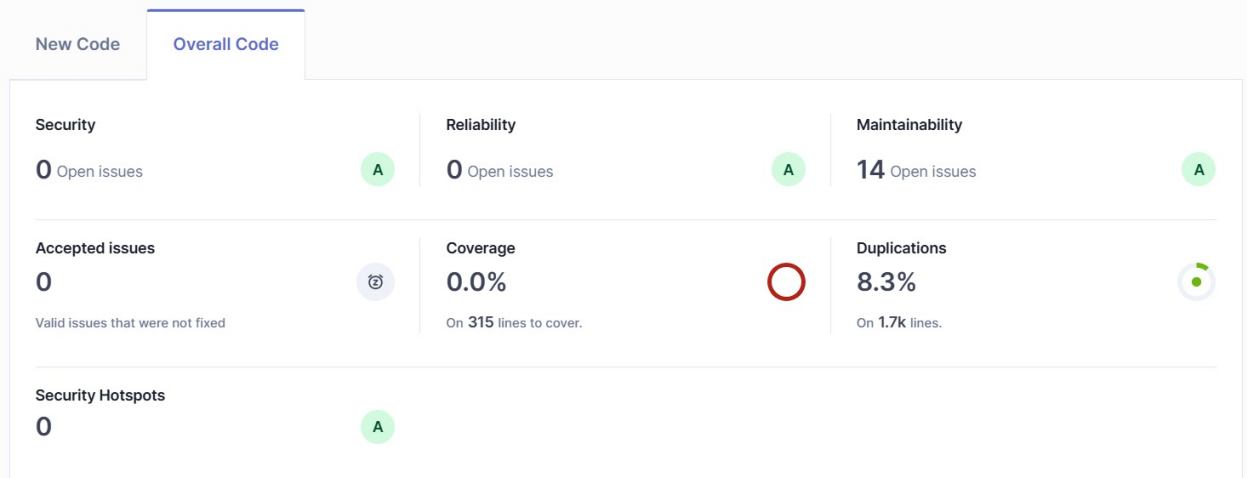
[+] src\App.tsx 0 problems			
[-] src\Filter.tsx 1 problem (1 error, 0 warnings)			
67:21	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
[+] src\Header.tsx 0 problems			
[+] src\account\Login.tsx 0 problems			
[+] src\account\SignUp.tsx 0 problems			
[+] src\account\api.tsx 0 problems			
[+] src\main.tsx 0 problems			
[+] src\proposal\AvailableProposals.tsx 0 problems			
[+] src\proposal\GiveFeedback.tsx 0 problems			
[+] src\proposal\PendingProposals.tsx 0 problems			
[+] src\proposal\ProposalReview.tsx 0 problems			
[+] src\proposal\ProposalSubmission.tsx 0 problems			
[+] src\proposal\api.tsx 0 problems			
[+] src\research\AvailableResearch.tsx 0 problems			
[+] src\research\ResearchDashboard.tsx 0 problems			
[+] src\research\api.tsx 0 problems			
[+] src\vite-env.d.ts 0 problems			

The backend of the client-server architecture shows efficient processing and routing. While performance metrics are not visualized here, the separation between frontend and backend contributes to better modularity and load distribution. However, SonarQube reports 9 reliability issues which could be related to architectural decisions or potential code smells.

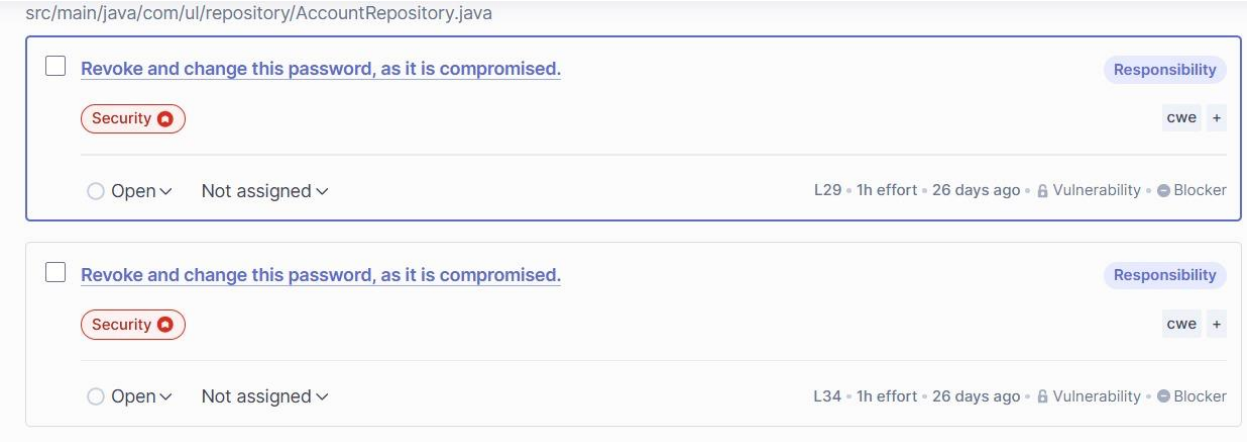


Security

The security assessment shows no critical vulnerabilities in the front-end, earning it an “A” grade in SonarQube. There are no open issues or security hotspots detected. This indicates adherence to best practices in terms of front-end web safety. However, it's worth noting that client-side security is inherently limited, so backend validation remains crucial.



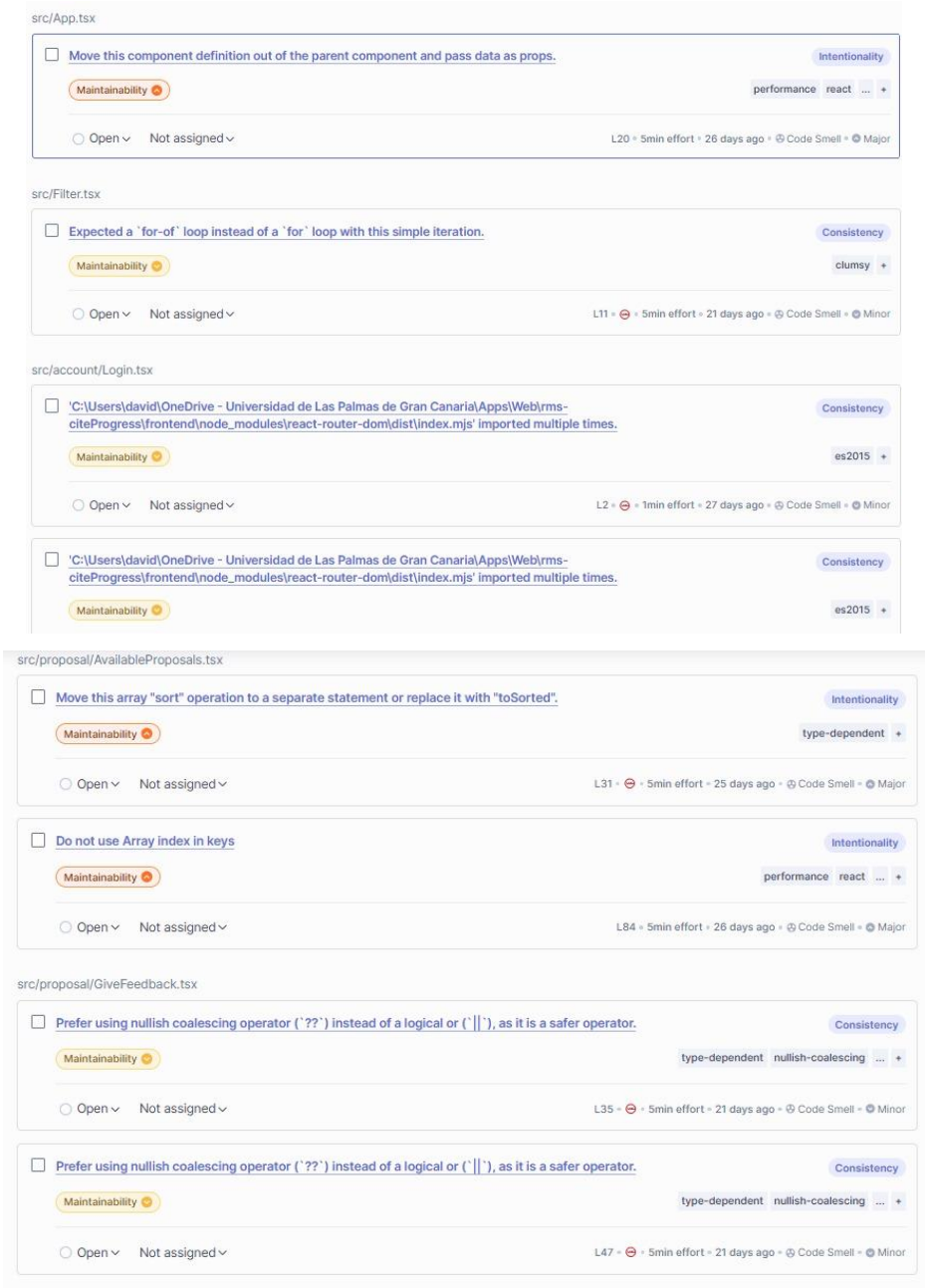
Unlike the MVC frontend, the backend presents two critical security issues. These relate to hardcoded passwords, which were flagged as blocker-level vulnerabilities. Additionally, two security hotspots were detected, highlighting the need for stronger credential handling and encryption mechanisms.

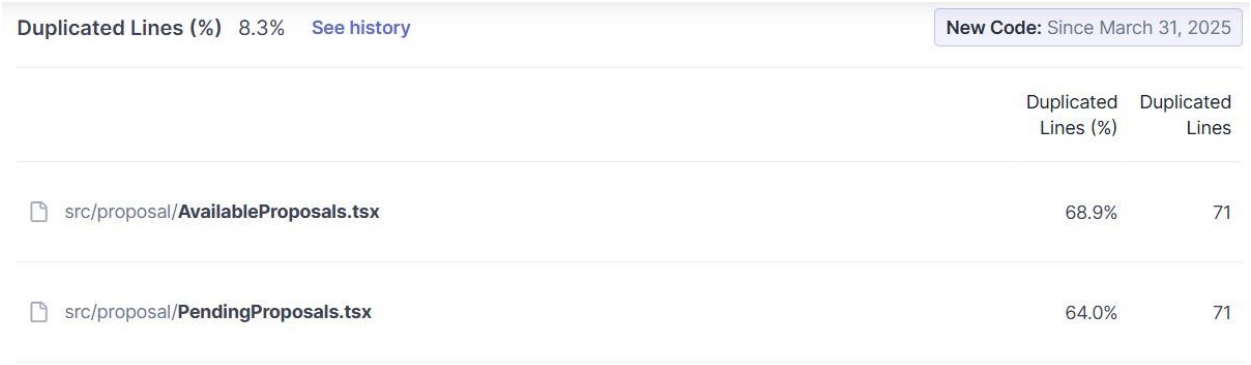


Maintainability

The front end has 14 open maintainability issues, with an overall “A” rating in SonarQube. Common issues include improper loop usage, poor component placement, deprecated practices,

and using array indexes as keys in React components. Additionally, some code duplications were detected in AvailableProposals.tsx (68.9%) and PendingProposals.tsx (64.0%), impacting maintainability and code efficiency.





Usability

The interface is clean, responsive, and thoughtfully designed with accessibility in mind. It offers a user-friendly experience through features such as form validation, role-based views, and intuitive navigation. These elements make the system easy to use and navigate for a wide range of users. However, there is still room for improvement, particularly in enhancing accessibility testing, implementing dark mode support, and providing clear user feedback on actions—such as success or failure notifications—to further enrich the overall user experience.

Microservices

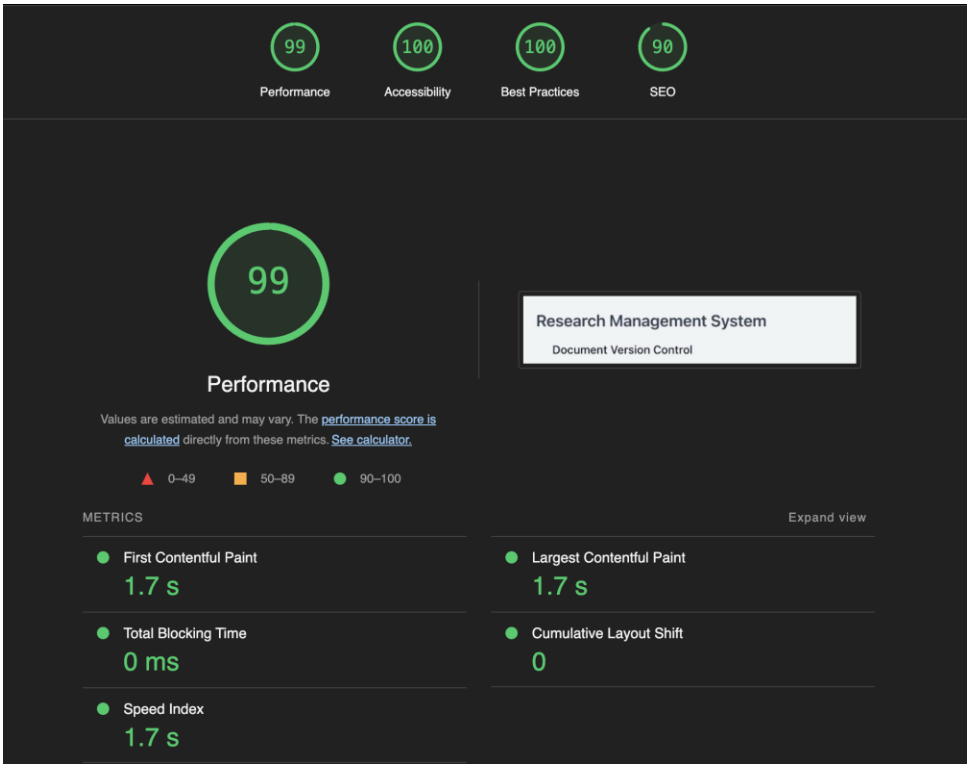


Figure 6 Performance of the Microservices Architecture

Performance is a strong suit of the prototype, with Lighthouse achieving a score of 99%. This indicates a highly optimized system that loads quickly and responds promptly. The report highlights minimal delays, and further optimization opportunities exist. Specifically, reducing unused JavaScript, eliminating render-blocking resources, and ensuring text compression can further enhance performance, leading to an even smoother user experience.

Security

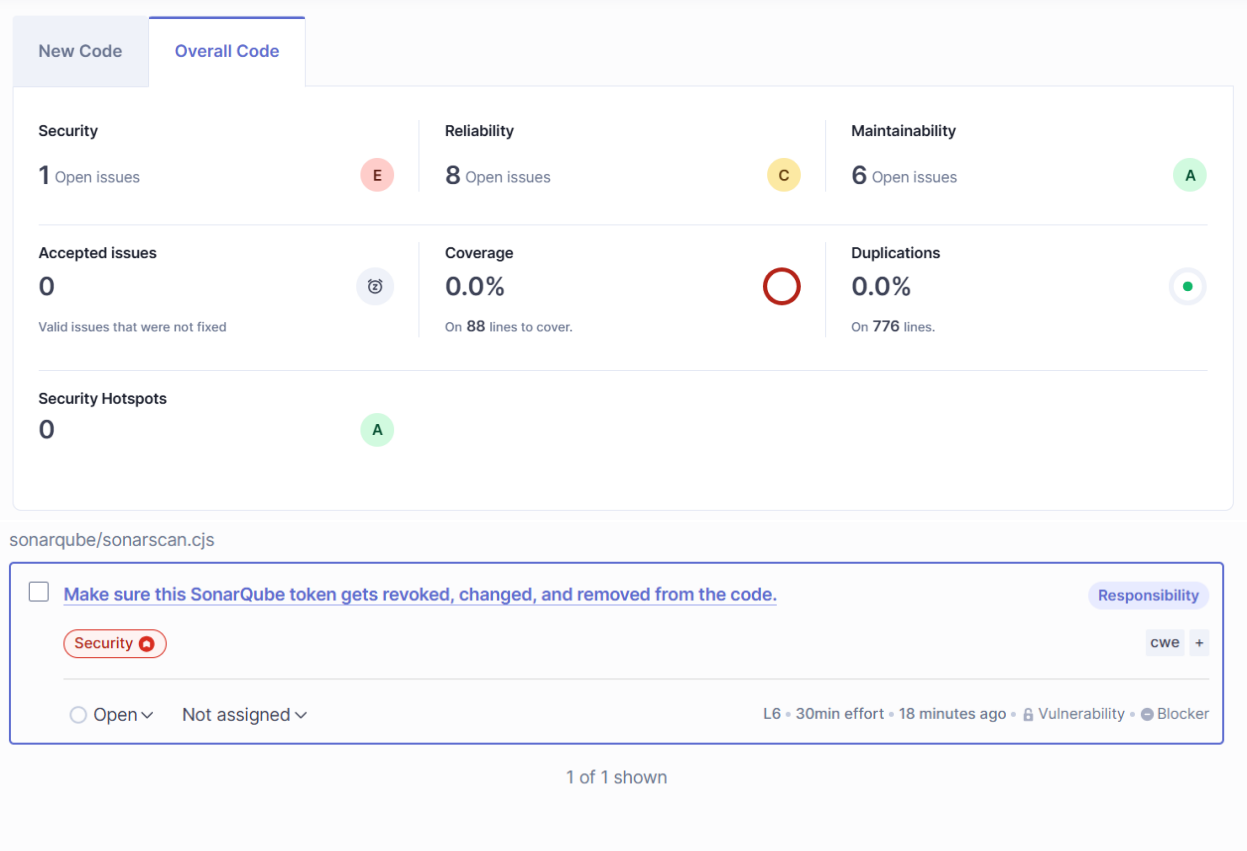


Figure 7 Security of the Microservices Architecture

Security is robust. The only issue identified is the SonarQube token which was added for the sole purpose of running this evaluation and was removed subsequently. There were no other reported issues identified during the SonarQube audit. This indicates a secure system, free from common vulnerabilities. The absence of identified security concerns suggests that best practices have been implemented to protect against potential threats.

Maintainability

eslint.html

Add the "let", "const" or "var" keyword to this declaration of "i" to make it explicit. — Intentionality

Maintainability — pitfall +

Open · Not assigned — L191 • 2min effort • 19 minutes ago • Code Smell • Blocker

Expected a `for-of` loop instead of a `for` loop with this simple iteration. — Consistency

Maintainability — clumsy +

Open · Not assigned — L195 • 5min effort • 19 minutes ago • Code Smell • Minor

src/App.css

Unexpected empty source — Intentionality

Maintainability — No tags +

Open · Not assigned — L1 • 1min effort • 8 minutes ago • Code Smell • Major

src/components/Header.tsx

Non-interactive elements should not be assigned mouse or keyboard event listeners. — Consistency

Maintainability · Reliability — accessibility react +

Open · Not assigned — L23 • 5min effort • 16 days ago • Code Smell • Minor

src/components/ResearchPaperSearch.tsx

Prefer using an optional chain expression instead, as it's more concise and easier to read. — Intentionality

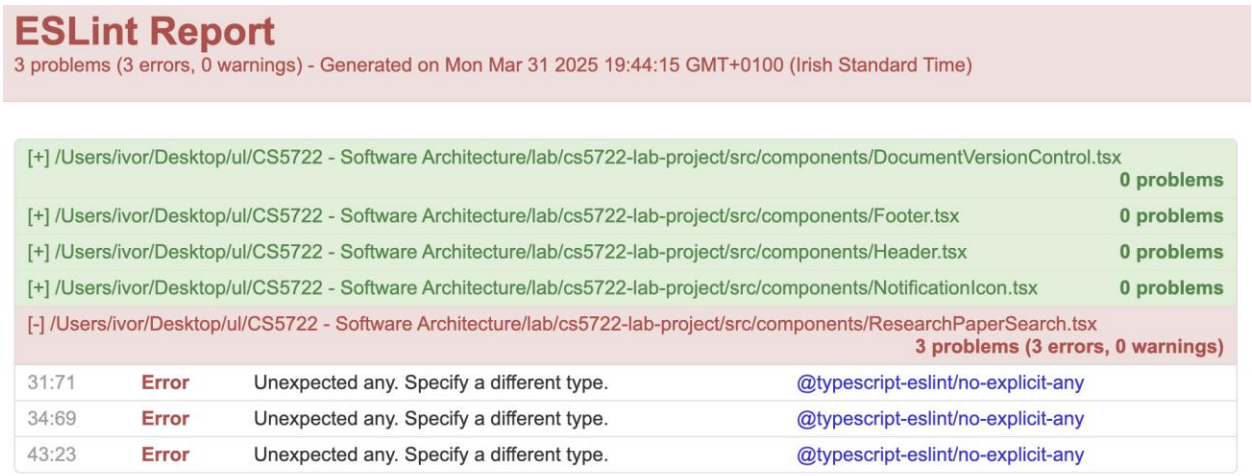
Maintainability — No tags +

Open · Not assigned — L30 • 5min effort • 16 days ago • Code Smell • Minor

Do not use Array index in keys — Intentionality

Maintainability — performance react ... +

Open · Not assigned — L73 • 5min effort • 27 days ago • Code Smell • Major



ESLint Report

3 problems (3 errors, 0 warnings) - Generated on Mon Mar 31 2025 19:44:15 GMT+0100 (Irish Standard Time)

[+] /Users/ivor/Desktop/ul/CS5722 - Software Architecture/lab/cs5722-lab-project/src/components/DocumentVersionControl.tsx			0 problems
[+] /Users/ivor/Desktop/ul/CS5722 - Software Architecture/lab/cs5722-lab-project/src/components/Footer.tsx			0 problems
[+] /Users/ivor/Desktop/ul/CS5722 - Software Architecture/lab/cs5722-lab-project/src/components/Header.tsx			0 problems
[+] /Users/ivor/Desktop/ul/CS5722 - Software Architecture/lab/cs5722-lab-project/src/components/NotificationIcon.tsx			0 problems
[-] /Users/ivor/Desktop/ul/CS5722 - Software Architecture/lab/cs5722-lab-project/src/components/ResearchPaperSearch.tsx			3 problems (3 errors, 0 warnings)
31:71	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
34:69	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
43:23	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any

Figure 8 Maintainability of the Microservices Architecture

Maintainability, while present, presents opportunities for improvement based on the code analysis findings. Several issues have been identified that, if addressed, could significantly improve the code's long-term health and ease of maintenance. Specific areas requiring attention include:

- Accessibility Concerns: The flagging of keyboard event listeners suggests a potential accessibility problem, which can make it difficult for users who rely on keyboards or assistive technologies to interact with the application.
- Code Readability and Conciseness: The recommendation to prefer optional chain expressions hints at areas where the code could be made more concise and easier to understand, increasing the maintainability of the codebase.
- Key Best Practices: Issues with the use of array indexes in keys. Also, the lack of explicit keywords in variable declarations point towards a need to refactor to conform with recommended best practices.
- Unintended Code: Code that is unused, or code that contains empty source

The code quality analysis revealed areas where the codebase could benefit from improvement, specifically impacting its maintainability. The ESLint report identifies three errors within the ResearchPaperSearch.tsx component. The reported errors are all of the same type: "Unexpected any. Specify a different type." These errors, flagged by the @typescript-eslint/no-explicit-any rule, indicate that the type any is being used in the code where a more specific type declaration would be preferable. This practice hinders type safety, making it difficult to catch potential errors during compilation and increasing the risk of runtime issues. This may lead to increased debugging time and a reduction in long-term code quality.

Usability

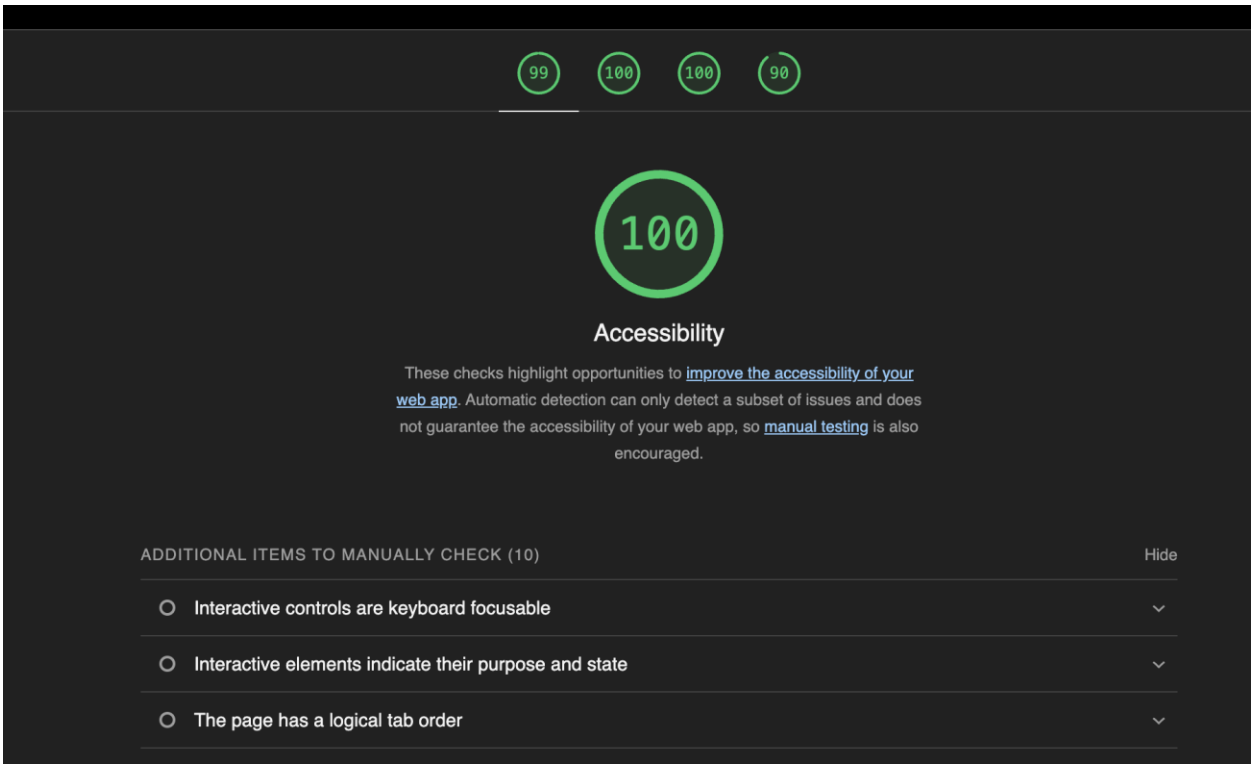


Figure 9 Accessibility of the Microservices Architecture

With a perfect Lighthouse score of 100%, the prototype demonstrates exceptional accessibility. This indicates that the system is designed to be usable and navigable by individuals with disabilities, adhering to a high standard of inclusive design. The robust accessibility score suggests that the application's features are well-structured, clearly labeled, and navigable via assistive technologies, including screen readers and keyboard navigation.

Hexagonal

Performance

Concerning performance, the hexagonal architecture prototype demonstrates exceptional loading speed, responsiveness, and stability. As shown in Figure 1, it achieves a perfect 100% performance score in Lighthouse testing, indicating optimal resource utilization and minimal delays. The key performance metrics are impressive, with First Contentful Paint and Largest Contentful Paint both at 0.2s, Total Blocking Time of 0ms, no Cumulative Layout Shift (0), and a Speed Index of 0.2s. The minimal main-thread work (0.1s) further demonstrates the efficiency of the application. These outstanding metrics indicate that the clear separation of concerns in the hexagonal architecture has not negatively impacted the front-end performance and may have contributed to the optimized asset delivery and processing. The architecture's ability to isolate

core business logic from external dependencies appears to benefit performance by allowing for more efficient code execution paths.

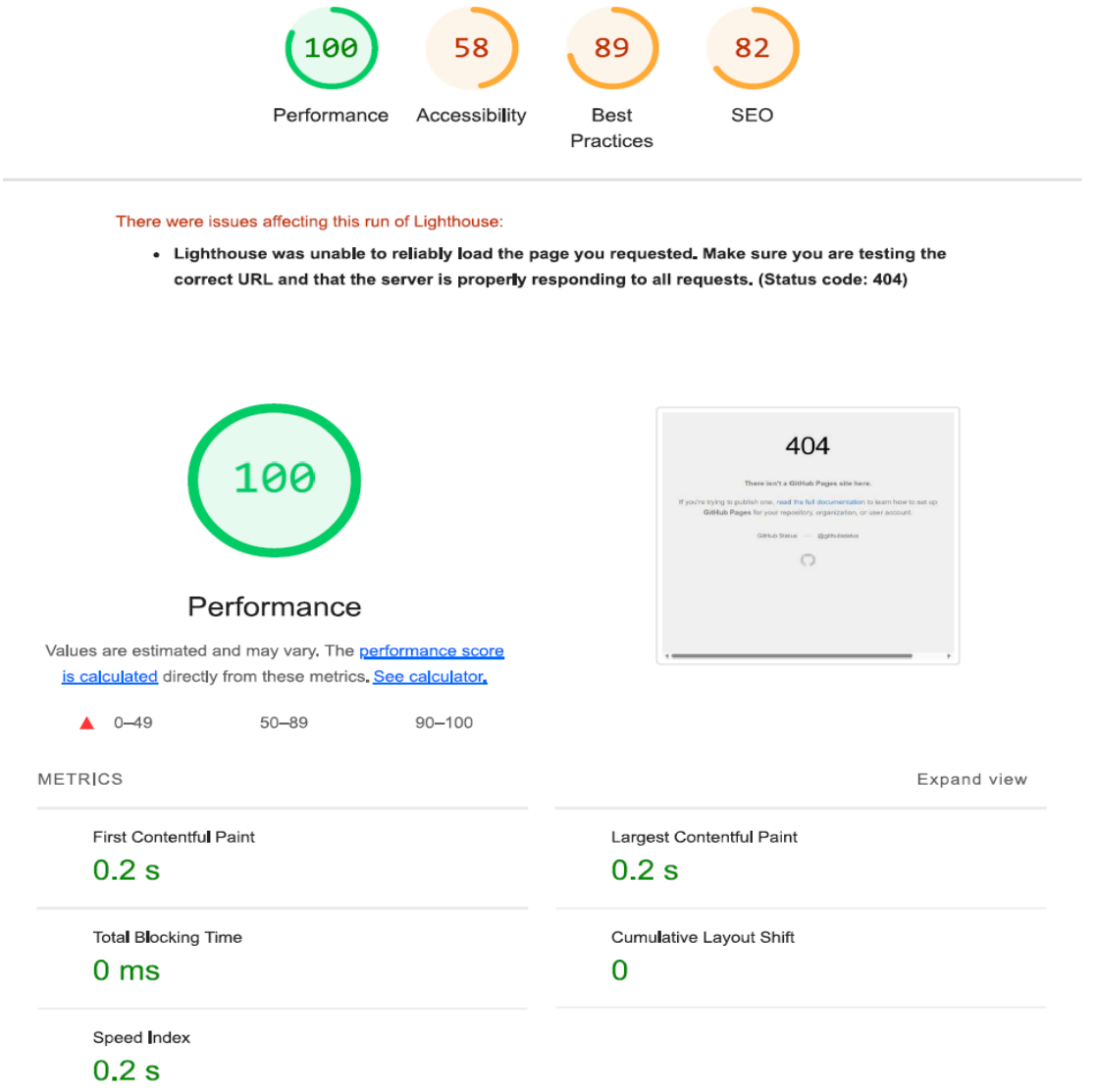
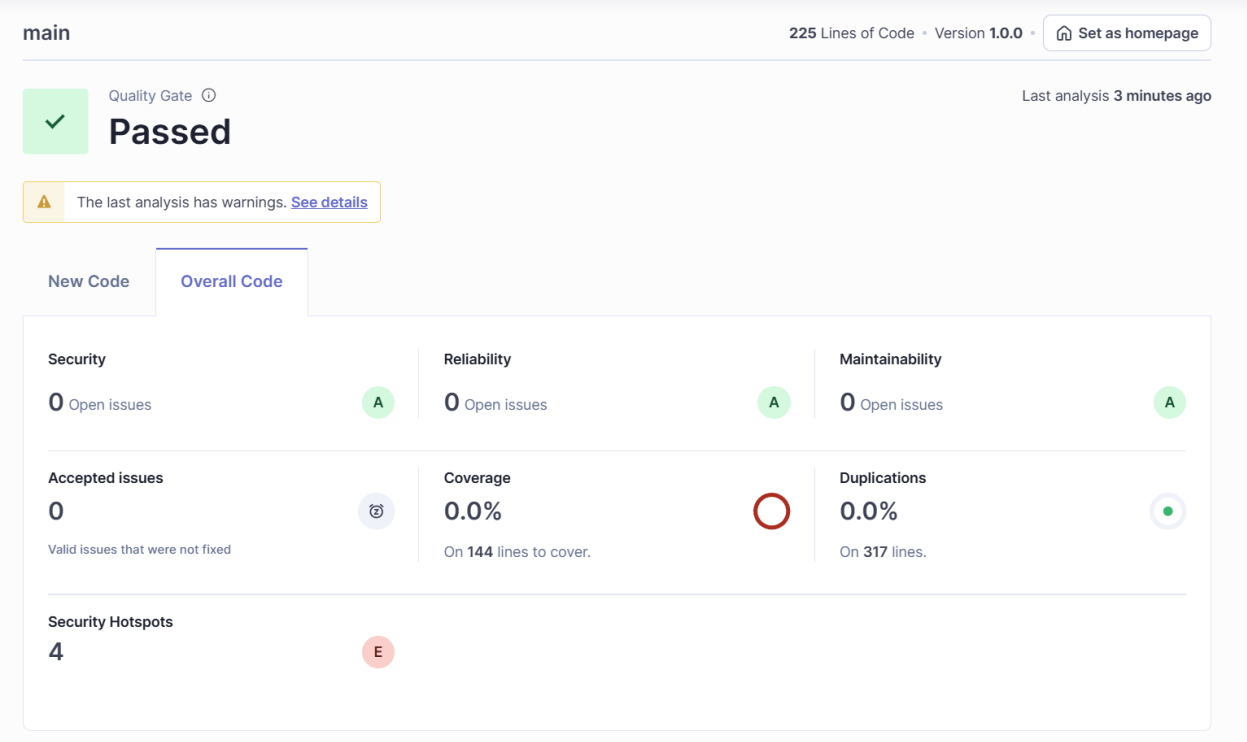


Figure 1 Performance of the Hexagonal Architecture

Security



As illustrated in the SonarQube report, the Hexagonal Architecture demonstrates a generally strong security posture, with 0 open security issues and an A rating. The system has passed the Quality Gate, indicating that it meets the basic security requirements established for the project. The architecture effectively implements JWT-based authentication, role-based access control (RBAC), and secure API calls to external services through dedicated security adapters.

However, SonarQube did detect 4 security hotspots that should be addressed in future iterations - most critically related to hard-coded passwords in the authentication service. The hotspot flags hard-coded credentials in the `services/AuthServiceSimple.js` file, which is marked as a high-priority issue that needs review. These hard-coded passwords are security-sensitive and could potentially be exploited if the source code is compromised, creating significant authentication vulnerabilities. The hexagonal architecture's clear separation between core business logic and external dependencies generally contributes significantly to the security posture by isolating authentication and authorization concerns in dedicated adapters. This architectural approach facilitates more consistent implementation of security controls and simplifies security auditing. However, the specific configuration issues identified by the security tools highlight that robust architecture alone cannot guarantee security without proper implementation practices.

Future iterations should focus on removing hard-coded credentials, implementing environment variables or a secure vault solution, applying proper HSTS and COOP headers, and addressing the identified dependency vulnerability to further strengthen the application's security posture.

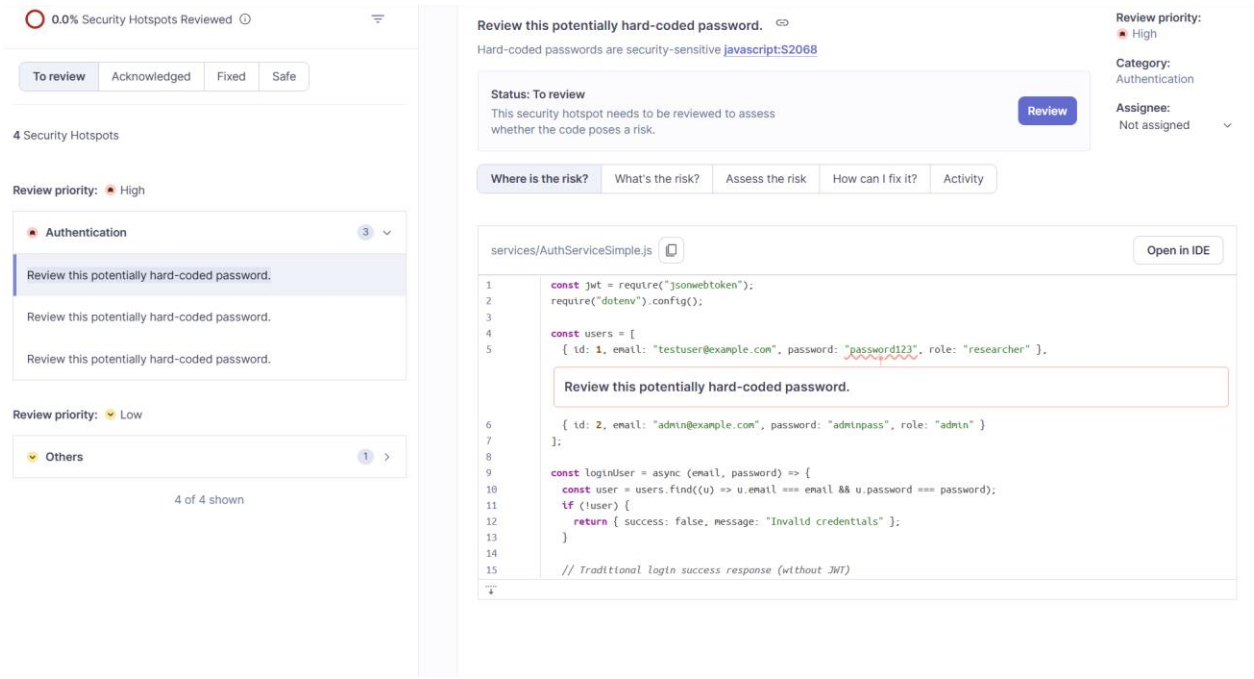


Figure 2: Security of the Hexagonal Architecture

Maintainability

Regarding maintainability, the ESLint analysis reveals several issues that should be addressed in future iterations. The system has approximately 854 ESLint errors, predominantly related to TypeScript implementation, unused expressions, and React-specific concerns. These issues can be categorized into:

1. TypeScript Implementation Issues:
    a. Extensive use of `no-unused-expressions` errors
    b. Usage of `any` type instead of more specific types
    c. Unused variables that could lead to code bloat
2. React Component Concerns:
    a. Missing prop validations
    b. React scope issues when using JSX
    c. Potential component lifecycle inefficiencies
3. Code Structure and Control Flow:
    a. Empty block statements
    b. Unexpected conditional assignments
    c. Unreachable code sections
    d. Missing break statements in case blocks

Despite these issues, the hexagonal architecture fundamentally enhances maintainability through:

- Clear component structure with separate UI, business logic, and infrastructure layers

- Well-defined interfaces between components through ports and adapters
- Enhanced testability through isolation of core business logic

This architectural approach creates a codebase that is more resilient to change, as modifications to one component (such as switching from one external API to another) can be implemented without affecting the core domain logic. However, addressing the identified ESLint warnings would significantly improve code quality and further enhance maintainability.



Figure 3 Maintainability of the Hexagonal Architecture (ESLint Report)

Usability

The usability assessment via Lighthouse reveals an accessibility score of 58, indicating moderate usability but with significant room for improvement. Key issues identified include:

1. Contrast Issues:
    a. Background and foreground colors do not have sufficient contrast ratio
    b. Links rely on color to be distinguishable
2. Navigation and Structure:
    a. Missing viewport meta tag, which may cause issues for mobile users
    b. Links without discernible names, making navigation difficult for screen reader users
    c. HTML element missing language attribute, affecting screen reader pronunciation
3. Document Structure:
    a. Missing meta description, which impacts SEO and user understanding of content

While the hexagonal architecture itself doesn't directly impact these frontends usability issues, the clear separation between the frontend presentation layer and the business logic provides potential for improved usability through:

- More consistent user interface behavior due to standardized adapter interfaces
- Potentially faster feature development through parallel work on front and backend components
- Improved reliability as core domain logic is protected from UI implementation details

To improve usability, future iterations should address the accessibility issues identified in the Lighthouse report, particularly focusing on color contrast, proper HTML structure, and navigation improvements.

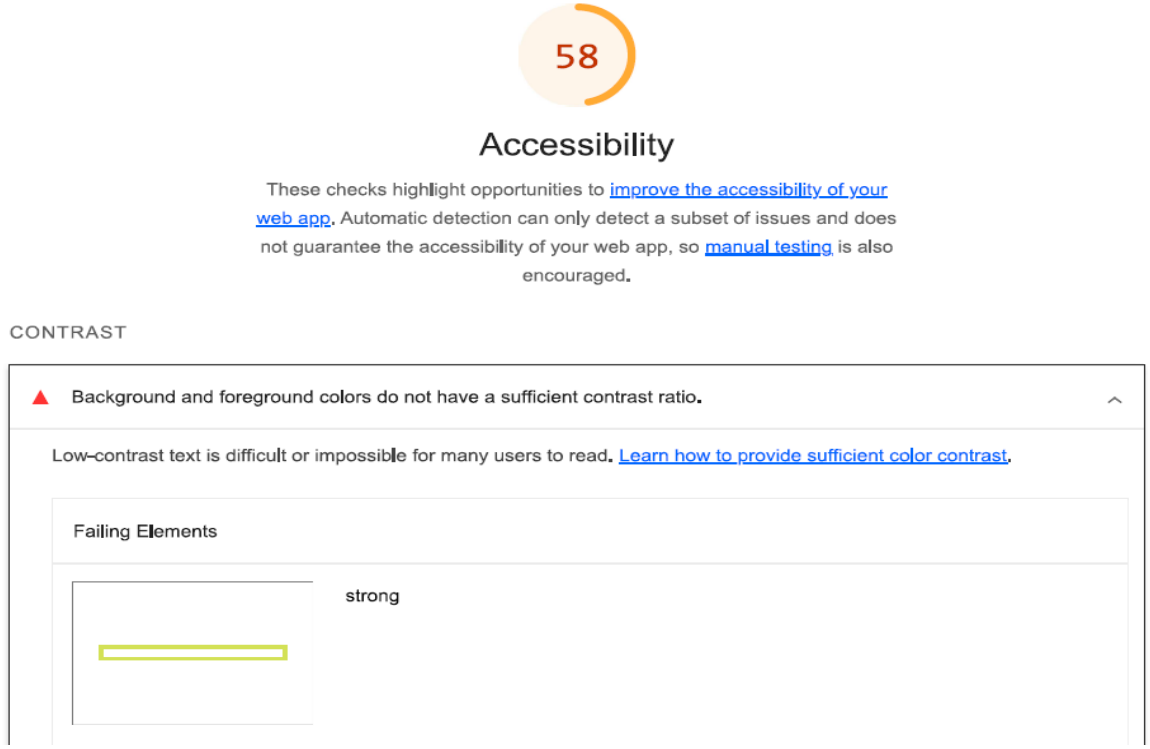


Figure 4: Usability of the Hexagonal Architecture

Client-Server Layered

Performance

Concerning performance, the Layered + Client-Server architecture demonstrates moderate loading speed, responsiveness, and stability. As shown in the Lighthouse report, it achieves a 60% performance score, indicating some areas for optimization. The metrics reveal a First Contentful Paint of 6.2s and a Largest Contentful Paint of 10.7s, which are relatively slow for optimal user experience. However, the architecture does show excellent Total Blocking Time (0ms) and perfect Cumulative Layout Shift (0), indicating good responsiveness once loaded. Based on the results obtained with Lighthouse, significant improvements could be made by enabling text compression (potential savings of 1,353 KiB), reducing unused JavaScript (potential savings of 821 KiB), and minifying JavaScript code (potential savings of 665 KiB). Additionally, reducing the 14 chains of critical requests would enhance the performance of the system.

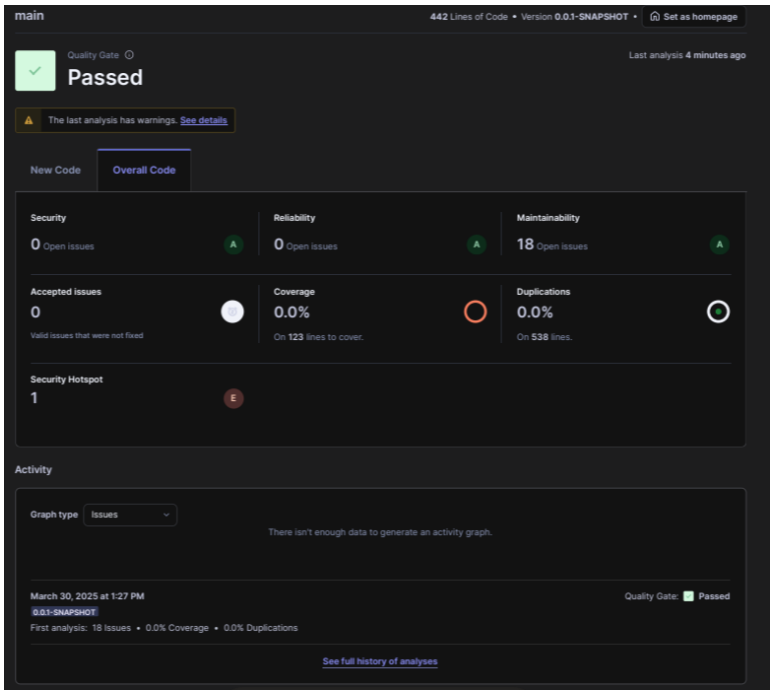


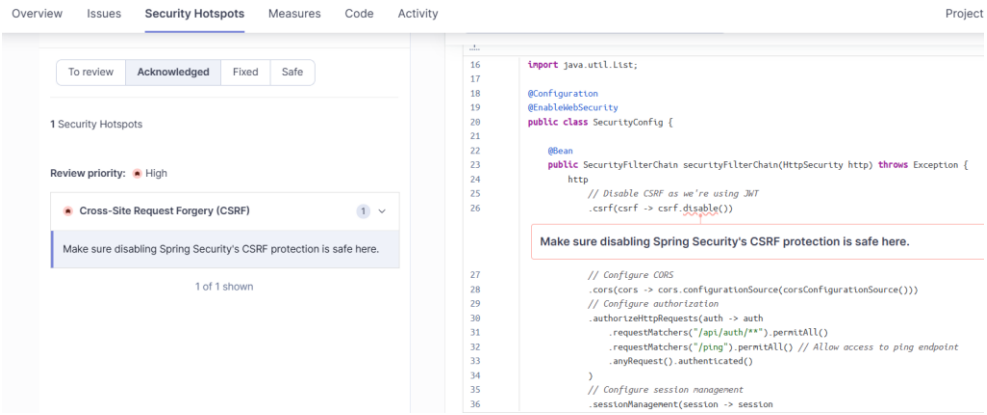
Security

As illustrated in the SonarQube report, the Layered + Client-Server architecture demonstrates strong security practices, with 0 open security issues and an A rating. The system also scores a perfect 100/100 in Lighthouse's Best Practices category, indicating that it is secure, reliable, and

adheres to web security standards. The architecture effectively implements protection against XSS attacks, uses a strong HSTS policy, ensures proper origin isolation with COOP, and mitigates clickjacking risks.

However, SonarQube did detect 1 security hotspot that should be addressed in future iterations - specifically related to Spring Security's CSRF protection being disabled. The hotspot flags the line "csrf.disable()" in the SecurityConfig class, which disables Cross-Site Request Forgery protection. This is marked as a high-priority issue that needs review, as disabling CSRF protection could potentially expose the application to forgery attacks where malicious sites could trick authenticated users into submitting requests without their knowledge. The layered architecture's clear separation between presentation, application, and data layers generally contributes significantly to security posture, but this specific configuration in the application layer represents a potential vulnerability that should be addressed.






```java
import java.util.List;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

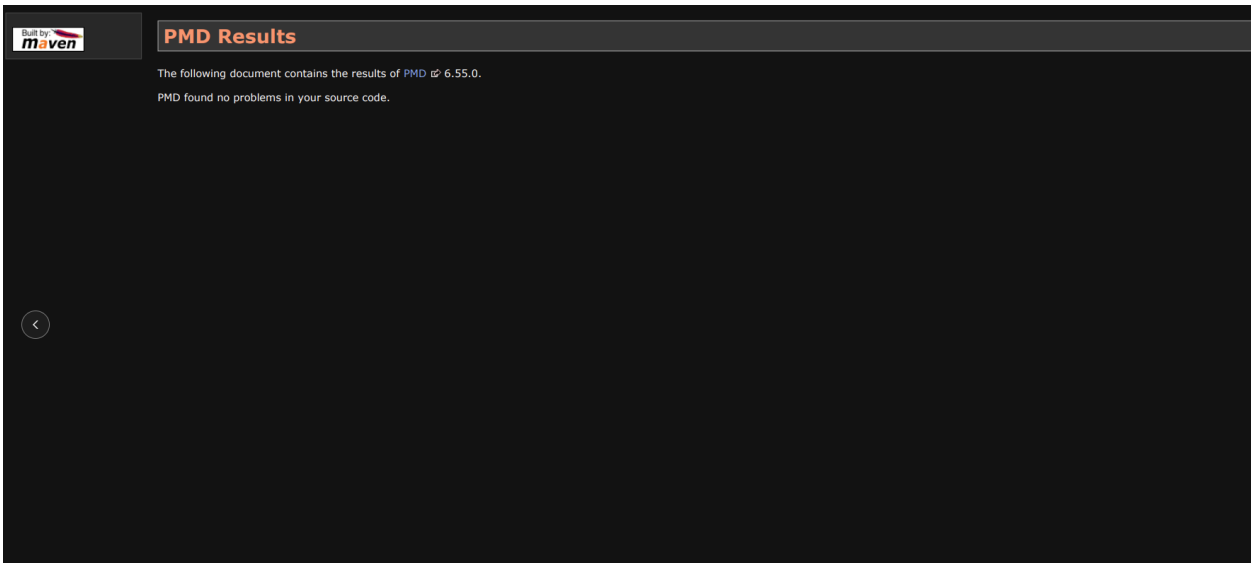
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // Disable CSRF as we're using JWT
            .csrf(csrf -> csrf.disable())
            // Configure CORS
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            // Configure authorization
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/ping").permitAll() // Allow access to ping endpoint
                .anyRequest().authenticated()
            )
            // Configure session management
            .sessionManagement(session -> session
```

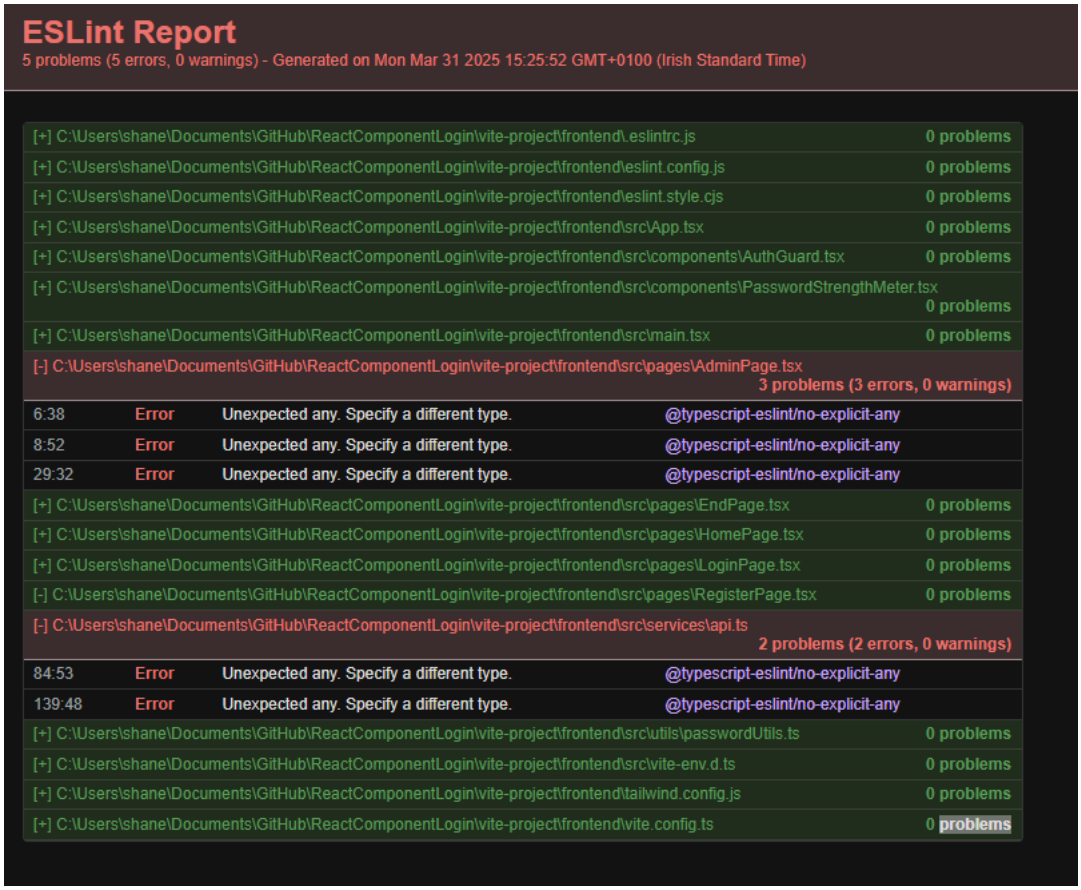
Maintainability

Regarding maintainability, the Layered + Client-Server architecture demonstrates good structural organization with clear separation of concerns. However, as illustrated in the SonarQube report, the system has 18 maintainability issues despite receiving an A rating overall. The backend Java code analyzed with PMD shows no issues, indicating well-structured Spring Boot implementation with proper layering of controllers, services, and repositories. The frontend React code, however, presents 5 issues detected by ESLint, specifically related to TypeScript type safety. The main concerns are:

1. **TypeScript Type Safety:**
    a. Unexpected 'any' data type usage in multiple files including AdminPage.tsx and api.ts
    b. Need to specify more precise types for stronger type checking

This discrepancy between backend and frontend quality highlights one of the challenges of the Layered + Client-Server architecture: maintaining consistent code quality standards across different technology stacks. The clear boundaries between layers, however, prevent these issues from cascading throughout the system.

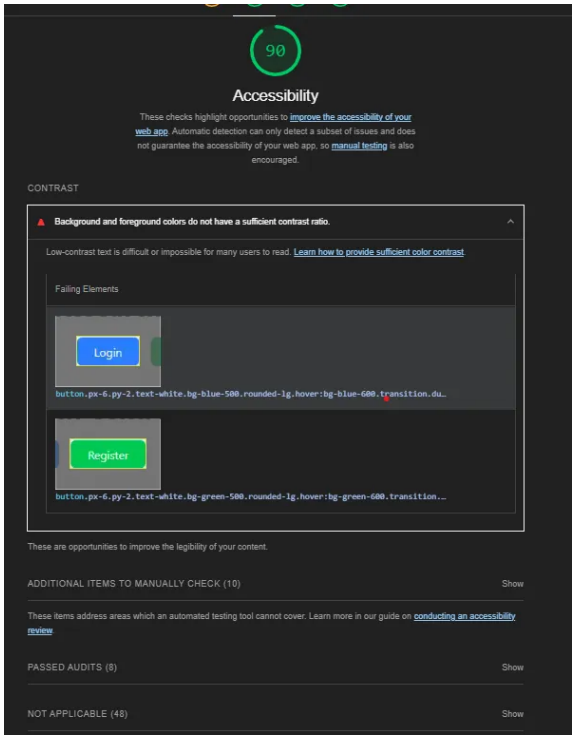

Built by maven
PMD Results
The following document contains the results of PMD 6.55.0.
PMD found no problems in your source code.


ESLint Report
5 problems (5 errors, 0 warnings) - Generated on Mon Mar 31 2025 15:25:52 GMT+0100 (Irish Standard Time)
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\.eslintrc.js 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\eslint.config.js 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\eslint.style.cjs 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\App.tsx 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\components\AuthGuard.tsx 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\components\PasswordStrengthMeter.tsx 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\main.tsx 0 problems
[-] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\pages\AdminPage.tsx 3 problems (3 errors, 0 warnings)
6:38 Error Unexpected any. Specify a different type. @typescript-eslint/no-explicit-any
8:52 Error Unexpected any. Specify a different type. @typescript-eslint/no-explicit-any
29:32 Error Unexpected any. Specify a different type. @typescript-eslint/no-explicit-any
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\pages\EndPage.tsx 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\pages\HomePage.tsx 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\pages\LoginPage.tsx 0 problems
[-] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\pages\RegisterPage.tsx 0 problems
[-] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\services\api.ts 2 problems (2 errors, 0 warnings)
84:53 Error Unexpected any. Specify a different type. @typescript-eslint/no-explicit-any
139:48 Error Unexpected any. Specify a different type. @typescript-eslint/no-explicit-any
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\utils\passwordUtils.ts 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\src\vite-env.d.ts 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\tailwind.config.js 0 problems
[+] C:\Users\shane\Documents\GitHub\ReactComponentLogin\vite-project\frontend\vite.config.ts 0 problems

Usability

After analyzing the Lighthouse report for this architecture, it can be concluded that it is highly usable and accessible, scoring 90/100 in both Accessibility and SEO categories. The well-structured separation between presentation and business logic layers contributes to a consistent user interface. The React frontend provides responsive components for the authentication functionality, with clear separation from the backend processing.

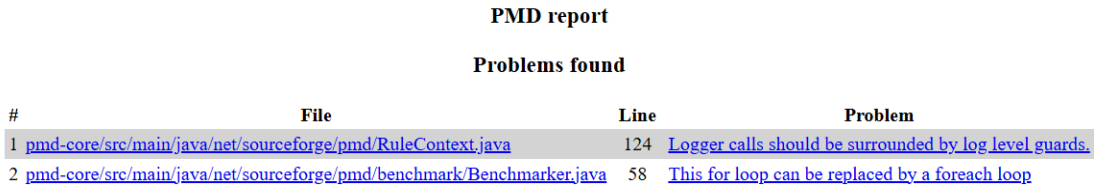
However, its accessibility could be improved, as the report highlights contrast ratio issues between background and foreground colors. Specifically, as shown in Figure 5, the login and register buttons have insufficient color contrast ratios. The blue login button and green register button both use light text colors that don't provide enough contrast against their backgrounds, making them difficult or impossible for many users to read. The layered architecture allows these UI improvements to be isolated to the presentation layer without affecting the underlying authentication logic in the business layer. Addressing these contrast issues, along with adding appropriate meta descriptions for SEO, would further enhance the usability of the system while maintaining the clean separation of concerns provided by the layered architecture.



3.3. Tools & Implementation

Regarding the tools used, four different tools were employed for the evaluation process:

PMD: A static code analyzer designed to identify potential issues in Java code, such as bugs, dead code, and performance problems. This tool was primarily utilized for the backend of projects that have a backend component in Java and was integrated as a Maven dependency. PMD was configured to generate HTML reports detailing the errors found.



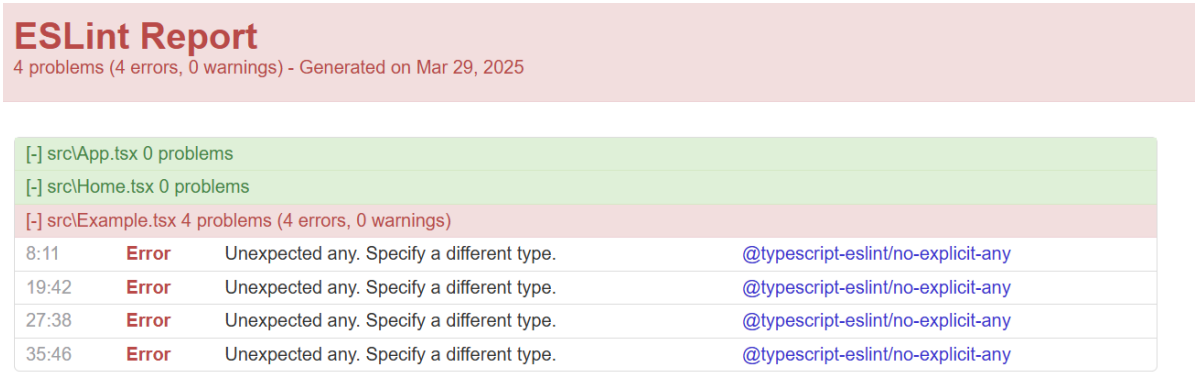
PMD report

Problems found

#	File	Line	Problem
1	pmd-core/src/main/java/net/sourceforge/pmd/RuleContext.java	124	Logger calls should be surrounded by log level guards.
2	pmd-core/src/main/java/net/sourceforge/pmd/benchmark/Benchmarker.java	58	This for loop can be replaced by a foreach loop

Figure 10 Example of a PMD report (PMD, 2025)

ESLint: A tool for identifying and fixing issues in JavaScript or TypeScript code, ensuring consistency, quality, and adherence to coding standards. ESLint was used for the front end of each prototype and integrated as a dependency in NPM. Like PMD, it was configured to generate HTML reports outlining the warnings and errors detected.



ESLint Report

4 problems (4 errors, 0 warnings) - Generated on Mar 29, 2025

[-] src\App.tsx 0 problems			
[-] src\Home.tsx 0 problems			
[-] src\Example.tsx 4 problems (4 errors, 0 warnings)			
8:11	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
19:42	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
27:38	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any
35:46	Error	Unexpected any. Specify a different type.	@typescript-eslint/no-explicit-any

Figure 11 Example of an ESLint report

SonarQube: A platform for continuous code quality inspection, offering reports on bugs, vulnerabilities, code smells, and metrics on code coverage and technical debt. SonarQube was used to empirically evaluate the quality attributes of each prototype and was incorporated using Docker for installation. It analyzes the project by running the tool to assess the code quality.

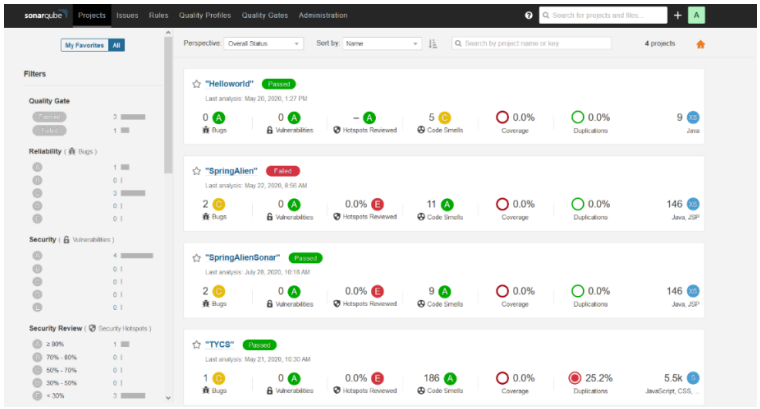


Figure 12 Example of SonarQube (Narsimha, 2020)

Lighthouse: An open-source, automated tool designed to enhance the quality of web pages. It was employed as an additional tool for evaluating the performance, accessibility, and SEO of projects that could be launched as websites. Lighthouse was included as a dependency in NPM and generates an HTML report of the website that will be evaluated.

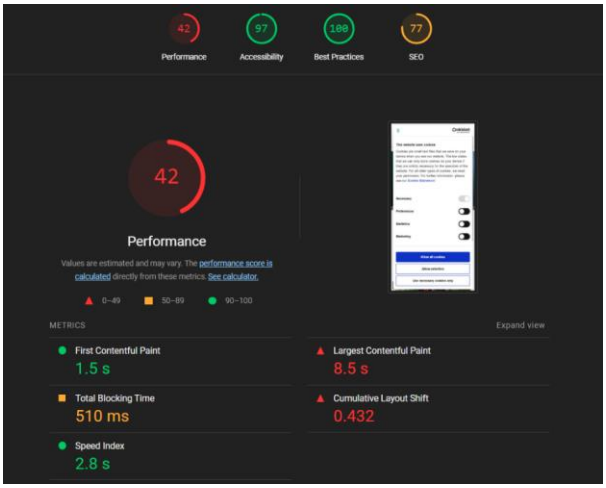


Figure 13 Report generated by Lighthouse

4. Quality Attributes Assessed

The assessment method employed for evaluating the quality attributes of the architectures combines automated tools with manual evaluation techniques. Performance was assessed using Google Lighthouse, a widely accepted tool for measuring web performance, while security was analysed using SonarQube, a static code analysis tool that identifies vulnerabilities. Similarly, maintainability was evaluated using SonarQube for detecting structural and readability issues, along with ESLint and PMD to identify Java and

TypeScript-specific problems. Usability was assessed through Lighthouse and manual testing, allowing for a thorough evaluation of accessibility and user experience.

Automated tools like Lighthouse, SonarQube, and ESLint provide precise, repeatable, and scalable measurements, making them ideal for assessing performance, security, and maintainability. However, manual testing was essential for usability evaluation, as aspects such as ease of navigation and interface clarity require human judgment. This hybrid approach ensures a comprehensive and well-rounded quality assessment of the Event-Driven prototype.

5. Conclusion

Rohan Conclusion

This particular implementation is a solid prototype which illustrates how research proposal handling flows intuitively and modularly. Good backend practices include the use of JWT authentication and a proper service-controller-repository separation. The UI adapts to user roles and offers different user experiences for Admins & Researchers. But this system isn't suitable for production due to the use of static in-memory data. There's no persistent storage, so things are lost on restart. Logging, test coverage, error handling, and encryption (HTTPS) are likewise absent. Integration with a database, complete authorization and authentication, and improvement of scaling & efficiency are crucial for production-readyness. However, CiteProgress achieves its objectives satisfactorily for demonstration, learning or early phase prototyping.

David Conclusion

Rohan’s prototype demonstrates strong performance, security, maintainability, and usability. With a high-performance score of 97%, it ensures efficient resource utilization. However, some strategies, such as reducing unused JavaScript and eliminating render-blocking resources, could enhance responsiveness. While no major security vulnerabilities are detected, the absence of a backend limits essential security features, and there is a SonarQube hotspot which should be considered. Maintainability is generally strong, but 28 issues related to code structure, readability, and best practices should be addressed for improved scalability. Finally, usability is high, offering an intuitive interface. Nonetheless, accessibility could be refined by improving list structures and heading hierarchies.

Ivor Conclusion

The Layered Architecture implemented by Shane with a Client-Server pattern provides a reasonable foundation for an authentication component. It offers a good balance between simplicity and functionality *for a prototype or small-scale application*. However, the reliance on in-memory data storage makes it unsuitable for production without significant modifications. Among the architectures considered, the implemented one is a good choice for the stated goals and constraints (simplicity, speed of development).

Aniruddh Conclusion

The hexagonal architecture implementation demonstrates excellent performance metrics and a reasonable security foundation through clear separation of concerns. While there are code quality issues that need to be addressed to improve maintainability and usability concerns that should be remediated, the fundamental architectural pattern provides a solid foundation for a maintainable, adaptable system. The architecture facilitates the isolation of core business logic from external dependencies, which enhances long-term flexibility and adaptability for future enhancements and integrations.

Shane Conclusion

This proposed implementation presents a well-designed application of Hexagonal Architecture that prioritizes modularity, maintainability, and adaptability. While some implementation aspects remain in progress, the architectural foundations are solid and support the system's continued evolution. The clear separation of core business logic from external dependencies positions the system well for future enhancements and integrations with additional research services.

6. Bibliography

Clements, P. C., 2002. *Software Architecture in Practice.* [Online]
Available at: https://www.researchgate.net/profile/Rick-Kazman/publication/224001127_Software_Architecture_In_Practice/links/02bfe510fef5da3230000000/Software-Architecture-In-Practice.pdf
[Accessed 29 March 2025].

Kazman, K., 2000. [Online]
Available at: http://www.nicodewet.com/assets/ATAM_Method_for_Architecture_Evaluation.pdf
[Accessed 29 March 2025].

Narsimha, 2020. *SonarQube for Continuous CodeQuality Testing.* [Online]
Available at: https://narsimhulu-464.medium.com/sonarqube-for-continuous-codequality-testing-3eecb6fe27e3
[Accessed 29 March 2025].

PMD, 2025. *PMD.* [Online]
Available at: https://github.com/pmd
[Accessed 29 March 2025].